Environmental Hacks 2026: Project Brief

Phone + PYNQ-Z2 build options · Prepared 10 October 2026 · Event window: 8–11 October 2026

Decision: build **BurnWatch**, an FPGA-accelerated open-burning smoke detector that turns any camera into a pollution-event sensor and uses AWS to verify, log and alert. If smoke detection is not reliable by **hour 8**, pivot to **GaugeEye**. It reuses the same camera ingest and AWS plumbing. **MeterEye** is a phone-only alternative that depends on access to a real water meter.

SELECTED

BurnWatch

Detects garbage/biomass burning smoke in a camera feed in real time. The FPGA analyses every pixel and only one suspect frame per event reaches the cloud.

Track: Air · FPGA: core · Phone: camera + alert receiver

FALLBACK (H8 PIVOT)

GaugeEye

A camera reads a striped gauge pole to measure waterlogging depth and rise rate. It raises a local alarm and sends alerts through AWS.

Track: Heat & Water · FPGA: optional · Phone: camera

ALTERNATIVE

MeterEye

An old phone watches the household water meter's low-flow indicator at night. If it moves while everyone sleeps, the house has a hidden leak.

Track: Heat & Water · FPGA: none · Phone: everything

Constraints we are designing for

- **Hardware on hand:** PYNQ-Z2 (Zynq-7020), ESP32-S3, smartphones, laptops. **No external sensors and no purchases.**
- **Rules:** new build during the event; AWS use must be visible in the demo; public repository; video of 3 minutes or less; written description. Judges assess the video, not a live demo.
- **Judging signals:** real-world impact, meaningful AWS use, usability for non-technical users, working execution, video quality. The organizer states that one working feature beats five incomplete ones and has warned against dashboard-only environmental projects.

Scoring (1 = poor, 5 = excellent; team judgment, not official scores)

Criterion	BurnWatch	GaugeEye	MeterEye
Real-world impact (Delhi relevance)	5	5	4
Feasibility in the remaining ~30 hours	3	5	4
FPGA justification	5	2	1
3-minute demo strength	4	5	2
Originality	5	4	3
Works with hardware on hand	4	5	3
Total / 30	**26**	**26**	**17**

BurnWatch and GaugeEye tie on total but differ in profile. BurnWatch has the higher ceiling: it is unique, the FPGA is integral, and it targets Delhi's most visible problem. GaugeEye has the higher floor: it is nearly impossible to fail on video. That is why GaugeEye is the hour-8 fallback. The earlier front-runner, PipePulse (pipe-mounted leak detector), was dropped because it needs a piezo contact sensor we do not have. A phone-microphone version is possible, but it loses the FPGA role.

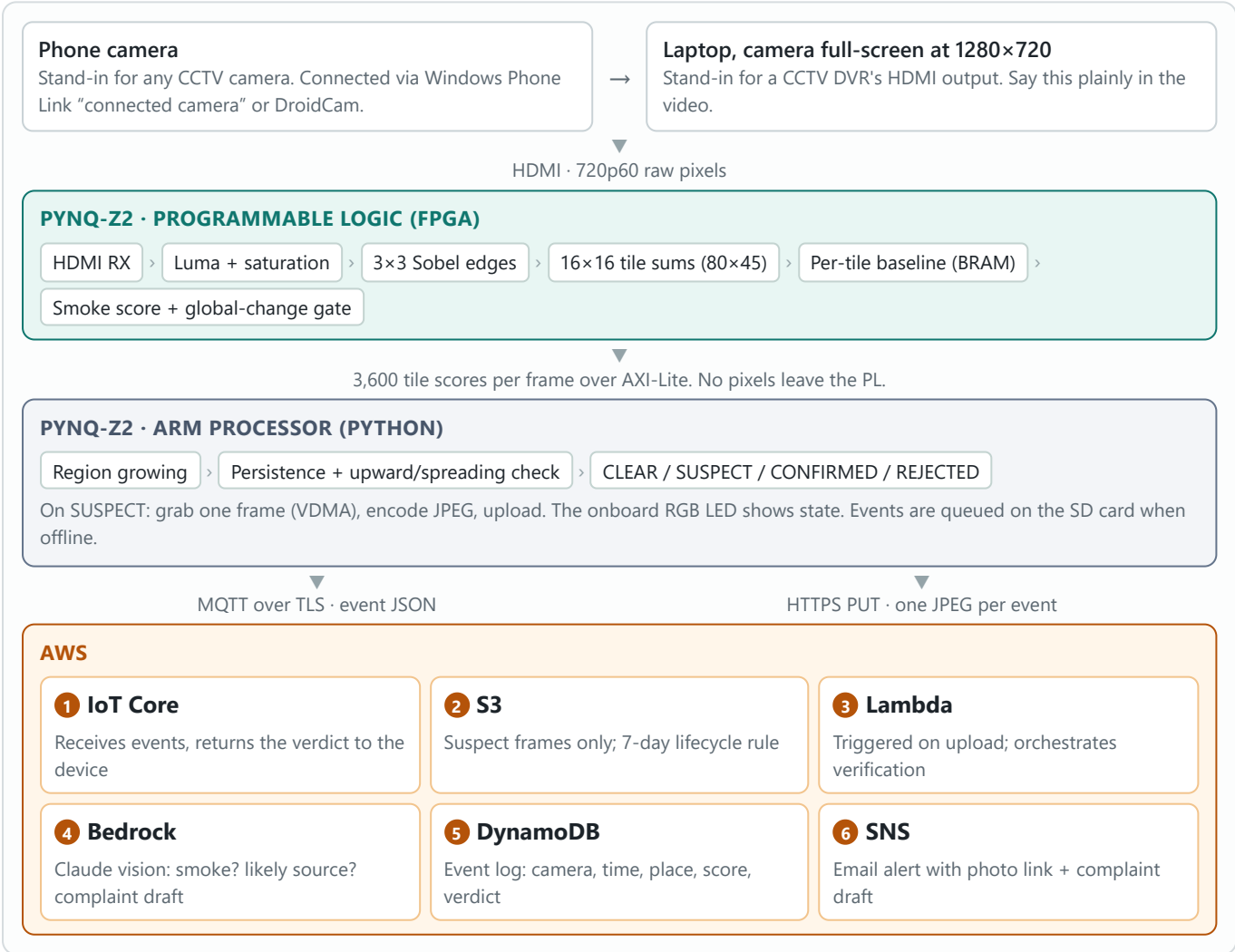
1 · BurnWatch SELECTED

Pitch: "Plug a box into any CCTV recorder's HDMI output and every camera becomes an open-burning sensor. The video never leaves the box. Only one verified frame per event does."

Problem and users

- Open burning of garbage, leaves and biomass is a recurring, prohibited local smoke source in Delhi, especially in winter, and enforcement relies on field teams. *Find a citable figure before using numbers in the video (see Verification notes).*
- Air-quality monitors report **how bad** the air is. They cannot say **where a burn is happening right now**, so nobody can act on it in time.
- **Users:** RWA and campus security, municipal field-enforcement teams, facility managers. They need *where* + *when* + *photo*, not a chart.
- **Why it is unique:** it detects a specific polluting *event* and routes it to someone who can stop it. This fits the organizer's "measurement to action" preference and avoids the AQI-dashboard trap.

Architecture



FPGA design: tile feature engine

Smoke softens the edges of whatever is behind it (high-frequency energy drops), it is low-saturation grey or white, and it changes slowly while rising and spreading. These are the classic video smoke cues from Töreyin et al. (EUSIPCO 2005,

wavelet-based real-time smoke detection), implemented here with integer arithmetic that maps cleanly to logic.

Stage	What it does	Rough cost
Video in	dvi2rgb + Video-In-to-AXI4-Stream, 1280×720 at 60 Hz, 24-bit RGB (from the PYNQ base overlay)	Existing IP
Colour features	Luma `Y = (77R + 150G + 29B) >> 8`; saturation proxy `max(R,G,B) − min(R,G,B)`	3 DSP, few LUTs
Edges	3×3 Sobel on Y with two line buffers; magnitude `|Gx| + |Gy|`	2 × BRAM18
Tile accumulators	Per 16×16 tile: Σedge, ΣY, Σsat. Only one tile-row (80 tiles) of accumulators is live at a time.	240 counters
Baseline	Per-tile slow EWMA of the three features (3,600 tiles): `b += (x − b) >> k`. **Frozen on flagged tiles** so smoke is not absorbed into the background.	1–2 × BRAM36
Score + gate	Score = edge drop vs. baseline × low saturation × luminance shift toward grey. If more than ~40% of tiles change at once (auto-exposure, lights, camera bump), ignore the frame and re-baseline faster.	Small
Readout	3,600-entry score map + frame counter, read by the ARM over AXI-Lite / BRAM port	AXI-Lite

Golden model: a NumPy implementation of exactly the same integer math. The PL score map must match it bit-for-bit on recorded frames. This is the verification story for the README and the video.

> **Honest FPGA claims.** (1) 720p60 is about 55 million pixels per second of per-pixel work. Measure the same algorithm on the ARM in Python versus the PL and show both numbers. (2) Raw video stays inside the box. Software sees 3,600 numbers per frame, and the cloud sees one frame per event. (3) A DVR's HDMI output is often a multi-camera grid, so the tile engine covers several cameras at once (future work). **Do not claim** low power, machine learning, or regulatory-grade measurement.

Device logic (ARM)

State	Entered when	Action
CLEAR	No persistent flagged region	Green LED; heartbeat every 60 s
SUSPECT	Connected flagged region persists for N seconds (start at 5–10 s, tune) and grows or drifts upward	Amber LED; one frame to S3 (max one per 2 min per camera); event to IoT Core
CONFIRMED	Cloud verdict says smoke with confidence above threshold	Red LED; SNS email sent by Lambda
REJECTED	Cloud verdict says not smoke (e.g., steam, fog, person)	Cooldown; the region is ignored for a few minutes

AWS details

```
Event (device → IoT Core, topic burnwatch/<camera>/events)
{ "camera": "rwa-gate-2", "state": "SUSPECT", "score": 0.71, "tiles": 46,
  "bbox": [312, 140, 520, 410], "s3_key": "frames/rwa-gate-2/1791640000.jpg",
  "ts": 1791640000 }

Verdict (Bedrock → Lambda → DynamoDB, SNS, and back to device)
{ "is_smoke": true, "confidence": 0.86, "likely_source": "open waste burning",
  "evidence": "grey plume rising from a ground-level pile next to the wall",
  "complaint_text": "Open burning observed at RWA Gate 2 at 21:14 ..." }
```

- **Model:** Claude (Haiku 5.5 for speed and cost) through Amazon Bedrock, temperature 0, JSON-only output. Enable model access at hour 0 and confirm regional availability (use a cross-region inference profile if needed). **Fallback verifier:** Amazon Rekognition DetectLabels, checking for a smoke label.

- **Alerts by email, not SMS:** SMS to Indian numbers through SNS requires TRAI DLT registration, which will not be done in a day.
- **Complaint text is a draft:** a human reviews it and submits it to their own channel (RWA, municipal helpline, or a government complaint app). Nothing is auto-submitted.
- **Device auth:** X.509 certificate for IoT Core; frame uploads via a pre-signed S3 URL or a tightly scoped IAM policy (PutObject on one prefix only).

3-minute video plan

Time	Shot
0:00–0:12	Hook, using your own footage: smoke rising from a burning pile. "Air monitors tell you the air is bad. They can't tell you where the burn is."
0:12–0:32	Setup: phone camera → laptop (DVR stand-in) → HDMI → PYNQ-Z2. "Every CCTV recorder has this HDMI port."
0:32–1:00	Light dhoop: the tile heat-map overlay lights up; state CLEAR → SUSPECT; RGB LED changes.
1:00–1:15	Confuser: a person walks through, or a grey cloth is waved. The state stays CLEAR.
1:15–1:50	AWS clearly on screen: IoT Core MQTT test client shows the event, then the S3 object, the Lambda log with the Bedrock JSON verdict, the DynamoDB row, and the SNS email arriving on the phone. State → CONFIRMED.
1:50–2:15	Architecture with measured numbers: "ARM only: X fps · FPGA: 60 fps · sent to cloud: one frame per event."
2:15–2:40	Test table (detected / missed / false alarms) and one honest limitation (steam).
2:40–3:00	Users: RWAs, campus security, field enforcement teams. Public repo link; "Built 8–11 October 2026".

Build plan (H0 = start of build)

Hours	Work	Exit check
H0–H1	New public repo, README skeleton, licences section. Check: HDMI cable, PYNQ image and matching Vivado/Vitis HLS version, AWS Bedrock model access, SNS email subscription.	Repo history starts during the event
H1–H3	Camera ingest: phone → laptop full-screen 720p → HDMI → base overlay `hdmi_in.readframe()` in Jupyter. Record test clips: empty scene, thin and thick dhoop smoke, person walking, grey cloth, kettle steam, lights on/off, camera bump.	Frames arrive on the PYNQ
H3–H7	Python fixed-point reference detector: tile features, baseline, score, global gate, persistence. Tune on clips and fill in the test table.	Smoke flagged; confusers mostly not
H8 gate	If smoke and confusers are not separable, **pivot to GaugeEye** and keep the ingest and AWS work.	Go / pivot
H8–H13	AWS: IoT Thing, certificate and policy; publish events; S3 upload; Lambda → Bedrock → DynamoDB → SNS; verdict back to the device.	End-to-end with the ARM-only detector
H13–H20	PL tile engine in Vitis HLS (or RTL): C-simulation against the golden model; insert into the base overlay video path (AXI4-Stream broadcaster tap); rebuild; AXI-Lite readout.	PL score map equals golden model
H20 cut	If the PL is not bit-exact, ship the ARM version. Present only the PL stages you have verified (e.g., Sobel + tile sums).	No unverified claims
H20–H23	Integration, offline queue, measure ARM fps vs. PL fps, rerun the test table.	Numbers for the video
H23–H25	README: problem, architecture, PL/PS split, AWS, test results, limitations, dependency credits. One architecture figure.	Repo is self-explanatory
H25–H29	Script, record (hardware take + AWS console take), edit to 3:00 or less with large captions.	Final cut
H29–H30	Upload, watch it end-to-end, verify the repo is public and links work, submit.	Submitted

Team of three: *FPGA owner* builds the HLS tile engine, rebuilds the overlay, matches the golden model and measures fps, starting at H3 in parallel. *Detector owner* handles clips, the Python reference detector, the state machine and the test table. *Cloud and story owner* handles the AWS stack, Bedrock prompt, SNS, README and video. **Solo:** follow the hour plan in order. PL work starts only after the ARM version works end to end.

Risks and mitigations

Risk	Mitigation
HDMI input does not lock	Set laptop output to 1280×720 at 60 Hz (duplicate display); try another cable. Last resort: phone → USB webcam mode / Wi-Fi → ARM → DMA into PL (weaker FPGA story).
Smoke too thin or low contrast	Close framing, plain darker background, several dhoop sticks, daylight.
Auto-exposure and lighting changes	Global-change gate; lock exposure if the camera app allows it.
Steam, fog, clouds look like smoke	Persistence check plus Bedrock verification; document steam as a known limitation.
Baseline slowly absorbs smoke	Freeze baseline updates on flagged tiles.
Long Vivado rebuilds	C-simulation first; start the PL track early; keep the ARM version always shippable.
Bedrock access or region issues	Request model access at H0; Rekognition fallback.
Laptop-in-the-middle looks fake	Say it plainly: the laptop plays the role of a CCTV DVR's HDMI output.

Demo safety. Burn only dhoop or agarbatti, or at most a small piece of paper in a steel bowl, outdoors, with a bucket of water within reach. **Never burn plastic or real waste for the demo.** It is toxic, and it contradicts the project.

Prerequisites checklist

☐ HDMI cable and a laptop with HDMI output (otherwise see the HDMI risk row above)
☐ Phone-as-webcam working: Windows Phone Link “connected camera” or DroidCam (laptop webcam as backup)
☐ PYNQ-Z2 image version noted; matching Vivado + Vitis HLS installed
☐ PYNQ-Z2 internet access for AWS (router Ethernet or laptop internet sharing)
☐ AWS account; Bedrock model access granted; SNS email subscription confirmed
☐ Dhoop/agarbatti, steel bowl, water bucket, plain background, phone stand

2 · GaugeEye FALLBACK (H8 PIVOT)

Pitch: “Paint a striped band on a pole at a waterlogging hotspot. Any camera that can see it becomes a water-depth sensor with a local alarm and AWS alerts.”

Problem and users

- Delhi's 2026 flood-control planning identified **169 locations** where waterlogging occurred the previous year, including 30 along Ring Road (news report cited in our scouting document).
- Demand is proven: Mumbai's BMC already runs **100 IoT waterlogging-depth systems** at chronic locations.
- **Users:** shopkeepers and guards at low points, campus/PWD maintenance, traffic police. They need “water is rising fast here”, not a map.

Architecture

Gauge pole
Ruler or stick with black/white bands every 1–5 cm, plus two calibration marks

→

Phone camera (fixed)
Same ingest as BurnWatch (HDMI to PYNQ), or processing directly in the phone browser

→

Edge logic
Waterline → depth → rise rate → GREEN / AMBER / RED; local alarm works offline

▼

MQTT over TLS · depth telemetry

AWS

IoT Core › IoT Rule › DynamoDB (depth series) + Lambda → SNS email on RED + Optional: S3 snapshot at RED

Algorithm

1. **Calibrate:** click the two marks once; this maps pixel rows to centimetres.
2. **Region of interest:** a narrow vertical strip around the pole.
3. **Band profile:** per row, measure band contrast (horizontal edge energy).
4. **Waterline:** the row where the periodic band pattern stops. Scan top-down and take the last row where the pattern is present.
5. **Clean up:** median of 5 frames; reject jumps larger than a few cm per frame; report a `quality` value.
6. **State:** depth and rise rate over 30–60 s feed GREEN → AMBER → RED with hysteresis on the way down.

```
{ "site": "model-drain-1", "depth_cm": 7.5, "rise_cm_per_min": 1.8,
  "state": "AMBER", "quality": 0.93, "ts": 1791640000 }
```

Use opaque water in the demo. In clear tap water the submerged bands stay visible and the waterline is ambiguous. Real floodwater is muddy, so add a spoon of soil, tea or turmeric. Also cover the gauge in clear tape (waterproof, matte) and angle the camera slightly downward to reduce glare.

Demo

Bucket or tray, gauge stick, phone on a stand. Pour water in stages: GREEN → AMBER → RED, alarm sounds, then IoT Core message, DynamoDB row and email. Drain the water to show the return to GREEN with hysteresis. Switch Wi-Fi off once to show the local alarm still works.

FPGA role (honest)

Weak. Strip-based waterline detection is light work for the ARM or even the phone. On a pivot from BurnWatch, keep the HDMI ingest and AWS plumbing and drop the PL tile engine. Expected effort: **about 14–18 hours** to a polished end-to-end result, so an H8 pivot leaves enough margin.

Risks

Risk	Mitigation
Glare and reflections	Matte tape, slight downward camera angle, avoid direct sunlight on the water.
Splashing and ripples	Median filter, jump rejection, quality flag; pour slowly at the edge.
Camera moves	Rigid mount; recalibrate if the marks shift; reject frames where the marks are lost.
Universal thresholds	Do not claim a universal “danger depth”. Thresholds depend on the site and are shown in the README calibration section.

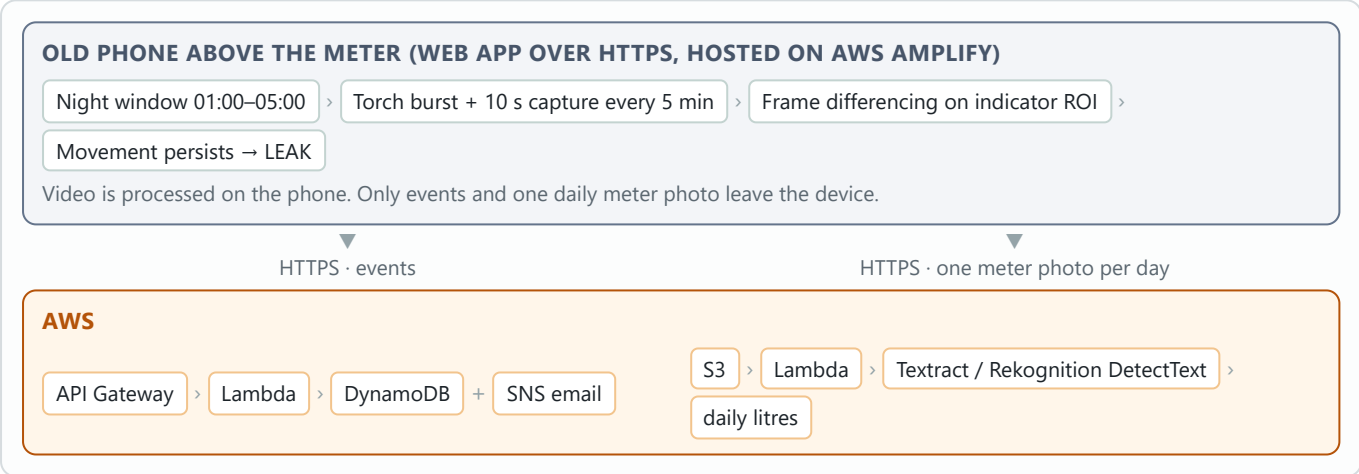
3 · MeterEye ALTERNATIVE

Pitch: "An old phone clipped over your water meter watches its low-flow indicator at night. If it moves while everyone is asleep, your house has a hidden leak."

Problem and users

- Hidden leaks in toilet cisterns, underground supply lines and tank float valves waste water continuously and go unnoticed for months.
- Water utilities find network leaks with **minimum night flow** analysis. MeterEye applies the same idea to one house, without touching the plumbing.
- Scale context: Delhi Jal Board serves about **19.5 million people** through roughly **14,355 km** of pipelines. The US EPA notes that a faucet dripping once per second can waste more than 3,000 gallons a year.
- **Users:** homeowners, hostel and building caretakers.

Architecture



Why it is interesting

- Detects whole-house hidden leaks with zero installation inside the plumbing.
- Reuses an old phone, so it adds an e-waste reuse angle.
- Daily OCR reading turns into consumption history at no extra hardware cost.

Why it ranks third

- **The demo needs a real, accessible water meter.** Without one the video is weak.
- No FPGA role at all.
- Camera-based meter reading already exists in hobby projects (ESP32-CAM meter readers), so it is less original.

Risks

Risk	Mitigation
No meter access	Confirm access before choosing this project. It is the deciding factor.
Night lighting, phone heat	Short torch bursts instead of a constant torch; phone on charger in a ventilated spot.
Fogged or dirty meter glass	Wipe and check daily; report a quality flag when the indicator is not visible.
Indicator designs vary	User marks the indicator region once during setup.
Browser stops the camera when the screen locks	Screen Wake Lock API; note that a native app is needed for real deployment.

4 · What we are not building

Idea	Why not
PipePulse (pipe-mounted leak sensor)	Needs a piezo contact sensor we do not have. A phone-mic version works but removes the FPGA role.
AQI / PM dashboard	The organizer explicitly warned about dashboard projects. We also have no PM sensor.
Smart-bin fill sensor	Crowded; a similar project recently won an India final.
FPGA CNN (FINN/Brevitas) as the centrepiece	The toolchain can consume the whole window before there is a product.
Flood-prediction ML, underground leak localization	Data and validation needs far exceed 30 hours. These add failure modes without improving the demo.

5 · Verification notes and reused components

Verify before the video / README

- Delhi open-burning figures: **none verified yet**. Find a citable source (e.g., Delhi Winter Action Plan, CAQM directions).
- 169 waterlogging locations and BMC's 100 IoT systems: from news reports in our scouting document. Re-check the links.
- DJB 19.5 million people and 14,355 km: from the scouting document (DJB).
- Töreyin et al., EUSIPCO 2005: confirm the citation details.
- EPA “one drip per second, over 3,000 gallons a year”: confirm on the EPA WaterSense page.
- Bedrock model availability and Rekognition smoke label: confirm in the AWS console for the chosen region.

Reused components (credit in README)

- `Xilinx/PYNQ` : overlays, HDMI in, MMIO, VDMA drivers (BSD-3-Clause)
- `aws/aws-iot-device-sdk-python-v2` : MQTT over TLS (Apache-2.0)
- `boto3` : S3 / Bedrock calls from Lambda (Apache-2.0)
- AMD Vitis Libraries, Vision (optional HLS building blocks) (Apache-2.0)
- NumPy (BSD-3-Clause), OpenCV (Apache-2.0)